

Lab 5: Non-linear Binary Classification using Neural Networks

Objective

The objective of this lab is to build a neural network capable of classifying data that is **not linearly separable**, and to understand why non-linear activation functions are necessary for this task.

Students will generate a non-linear 2D dataset, build a multi-layer neural network using PyTorch's `nn.Module`, train it with `BCEWithLogitsLoss` and SGD, and visualize the model's decision boundary before and after training.

Problem Statement

- Simple linear models (like the one from Lab 4) can only draw a straight-line decision boundary between classes.
- Many real-world datasets, however, are not linearly separable i.e. a straight line cannot correctly separate the classes.
- This lab uses the "moons" dataset which has two interleaving crescent shapes as an example of such a non-linearly separable binary classification problem. Given features X (2D coordinates) and labels y (0 or 1), the goal is to train a neural network that learns a curved (non-linear) decision boundary to correctly classify the points.
- To achieve this, the network stacks multiple linear layers with ReLU non-linear activation functions in between, since stacking linear layers alone (without a non-linearity) would still collapse into an equivalent single linear function.
- Training uses Binary Cross-Entropy loss (with logits) and Stochastic Gradient Descent, optimizing the network's weights to minimize classification error while tracking both loss and accuracy on training and test data.

Step 1: Import PyTorch and Check Version

```
import torch
print(torch.__version__)
```

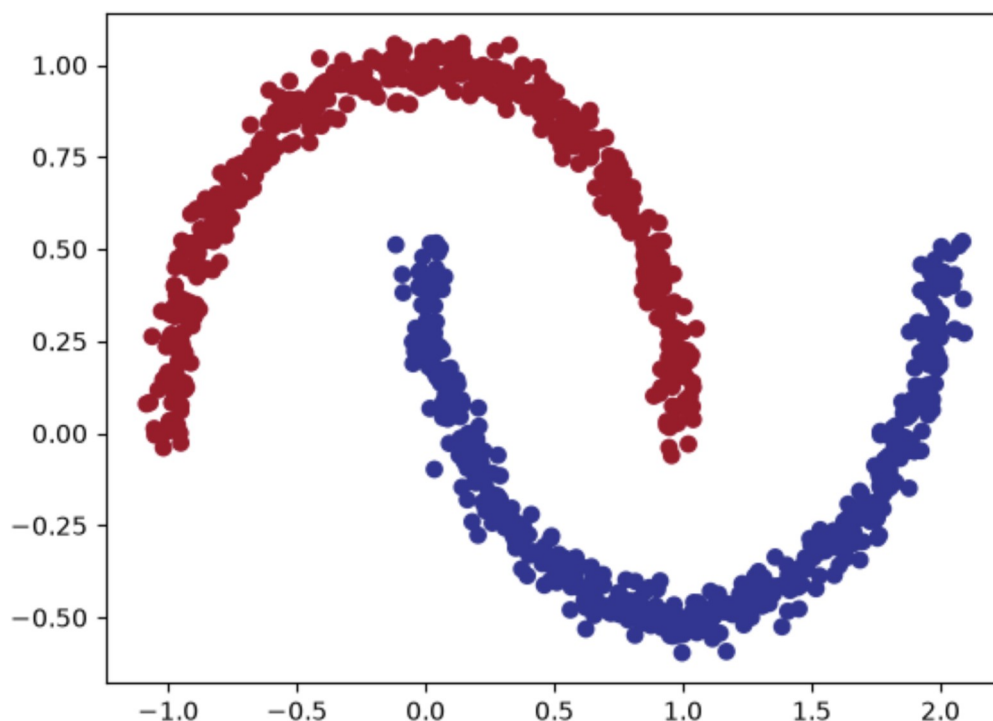
Step 2: Check Available Device

```
if torch.cuda.is_available():
    device = "cuda"
elif torch.backends.mps.is_available():
    device = "mps"
else:
    device = "cpu"
print(f'Using device ={device}')
```

Step 3: Generate the Dataset

```
from sklearn.datasets import make_moons
import matplotlib.pyplot as plt
n_samples = 1000
X, y = make_moons(n_samples=n_samples, noise=0.04, random_state=42)

plt.scatter(X[:,0], X[:,1], c=y, cmap=plt.cm.RdYlBu);
```



Step 4: Convert to Tensors and Train-Test Split

```
from sklearn.model_selection import train_test_split
X = torch.from_numpy(X).type(torch.float)
y = torch.from_numpy(y).type(torch.float)

X_train, X_test, y_train, y_test =
    train_test_split(X, y, test_size=0.2, random_state=42)
```

Step 5: Define the Model Class

```
from torch import nn

class CircleModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer_1 = nn.Linear(in_features=2, out_features=10)
        self.layer_2 = nn.Linear(in_features=10, out_features=10)
        self.layer_3 = nn.Linear(in_features=10, out_features=1)
        self.relu = nn.ReLU()

    def forward(self, x):
        return self.layer_3(self.relu(self.layer_2(self.relu(self.layer_1(x)))))
```

Step 6: Instantiate the Model and Inspect Parameters

```
model = CircleModel()
print(model.state_dict())
```

Step 7: Define Loss Function and Optimizer

```
loss_fn = nn.BCEWithLogitsLoss()
optimizer = torch.optim.SGD(params=model.parameters(), lr=0.05)
```

Step 8: Define an Accuracy Function

```
def accuracy_fn(y_true, y_pred):
    correct = torch.eq(y_true, y_pred).sum().item()
    acc = (correct/len(y_true))*100
    return acc
```

Step 9: Training Loop

```
epochs = 2500
train_loss_list = []
```

```
test_loss_list = []
torch.manual_seed(42)
# Cast the model and data to the device
model.to(device)
X_train, y_train = X_train.to(device), y_train.to(device)
X_test, y_test = X_test.to(device), y_test.to(device)

for epoch in range(epochs):
    ### TRAINING
    model.train()

    # Forward Pass
    y_logits = model(X_train).squeeze()
    y_pred = torch.round(torch.sigmoid(y_logits))

    # Compute Loss
    loss = loss_fn(y_logits, y_train)
    acc = accuracy_fn(y_true=y_train, y_pred=y_pred)

    # Optimizer Zero Grad
    optimizer.zero_grad()

    # Backpropagate
    loss.backward()

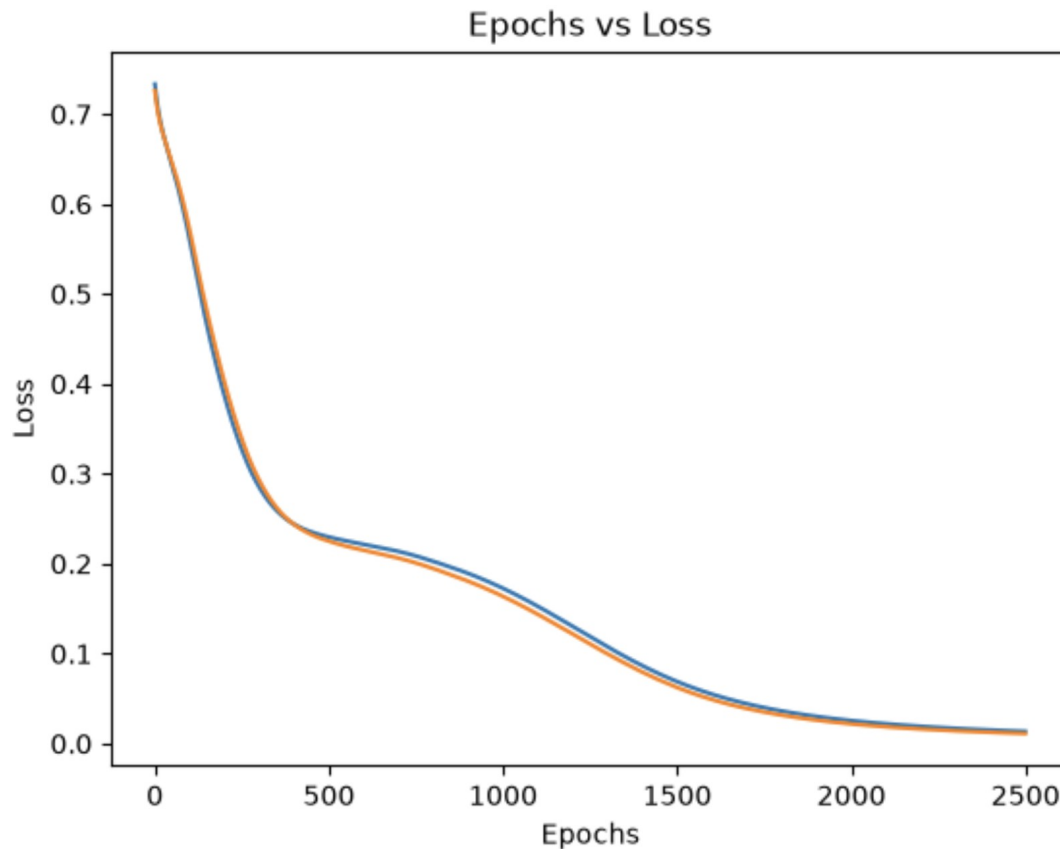
    # Step the optimizer
    optimizer.step()

    ### TESTING
    model.eval()
    with torch.inference_mode():
        test_logits = model(X_test).squeeze()
        test_preds = torch.round(torch.sigmoid(test_logits))
        test_loss = loss_fn(test_logits, y_test)
        test_acc = accuracy_fn(y_true=y_test, y_pred=test_preds)

    train_loss_list.append(loss.item())
    test_loss_list.append(test_loss.item())
    if (epoch+1) % 100 == 0:
        print(f'Epoch {epoch+1}|Loss {loss:.5f}|Acc {acc:.2f}|Test
Loss {test_loss:.5f}|Test Acc {test_acc:.2f}')
```

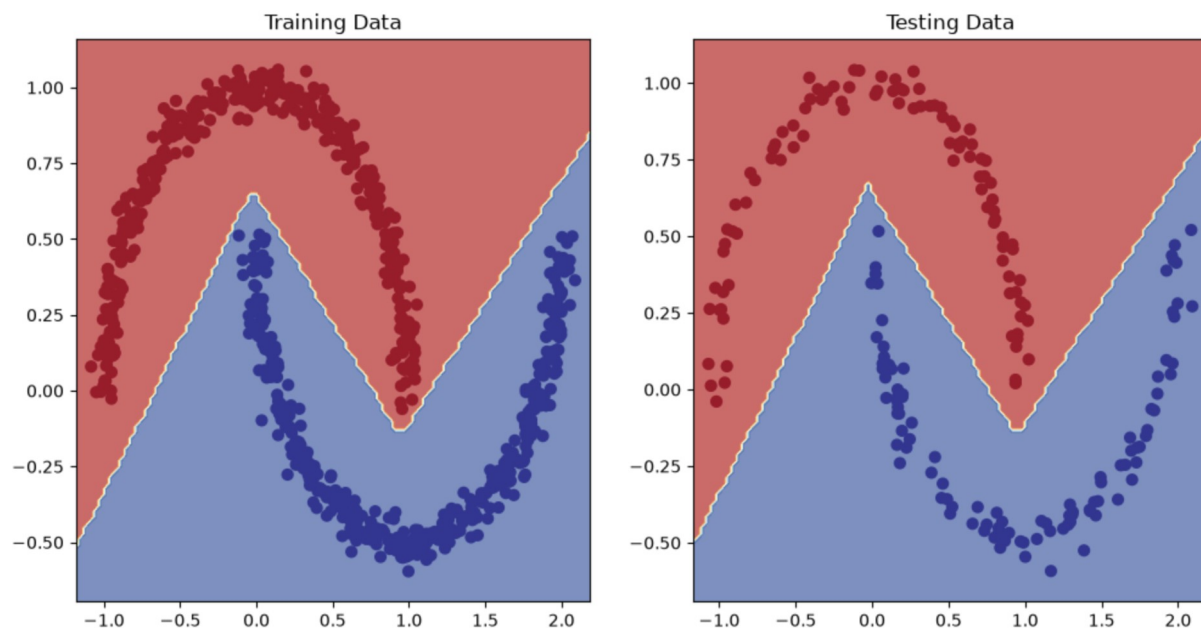
Step 10: Plot Loss Curves

```
plt.plot(range(epochs), train_loss_list, label="Training Loss")
plt.plot(range(epochs), test_loss_list, label="Testing Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.title("Epochs vs Loss");
```

**Step 11: Visualize the Decision Boundary**

```
from helper_functions import plot_decision_boundary

plt.figure(figsize=(12,6))
plt.subplot(1,2,1)
plt.title("Training Data")
plot_decision_boundary(model, X_train, y_train)
plt.subplot(1,2,2)
plt.title("Testing Data")
plot_decision_boundary(model, X_test, y_test)
```



Tasks

- Implement the `CircleModel` neural network as shown, train it, and record the final training/testing loss and accuracy.
- Plot the decision boundary before and after training to visually confirm the model has learned a non-linear boundary.
- Experiment with different learning rates (e.g., 0.01, 0.05, 0.1, 0.5) and compare their effect on convergence speed and final accuracy.
- Experiment with the number of hidden units in `layer_1/layer_2` (e.g., 5, 10, 20, 50) and observe the impact on decision boundary smoothness and accuracy.
- Replace `ReLU` with another activation function (e.g., `nn.Tanh` or `nn.Sigmoid`) and compare training behavior and final decision boundary.
- Replace the optimizer with `torch.optim.Adam` and compare its convergence speed and stability against SGD.
- Add an extra hidden layer to the network and evaluate whether it improves performance on this dataset.
- Repeat the experiment using `make_circles` instead of `make_moons` and compare the resulting decision boundary.
- Save the trained model's `state_dict`, reload it into a new model instance, and verify predictions match the original model.